High-Level Development Workflow

Project: **SMART-STOCK**
Role: **Software Developer**
Purpose: Define the development process used by the SmartStock team for implementing, reviewing, and integrating system changes

1. Development Workflow

The SmartStock development process follows a ticket-based workflow. Development starts from a refined Jira ticket and continues through development, testing, review, demonstration, integration, and QA verification.

Development Process

Step	Activity	Description
1	Jira Ticket	Development starts with a refined Jira ticket containing the requirement and acceptance criteria.
2	Requirement Review	The developer reviews the requirements and acceptance criteria before development.
3	Create Branch	The developer creates a working branch from `develop` for the assigned ticket.
4	Development	The developer implements the assigned feature, bug fix, or maintenance task.

5	Local Testing	The developer tests the changes locally before requesting review.
6	Commit Changes	Changes are committed using precise and concise commit messages.
7	Pull Request	The developer opens a Pull Request against `develop`.
8	Code Review	The implementation is reviewed for correctness, tests, security, data, and technical design.
9	SA Final Review	The Software Architect performs the final technical review.
10	Feature Demonstration	The developer demonstrates the implemented feature or change.
11	Merge	Approved changes are squash-merged into the integration branch.
12	QA Testing	QA verifies the implementation against the requirements and acceptance criteria.

The reference workflow follows a **ticket-first** process, requires a branch from `develop`, and proceeds through Pull Request review before merging.

2. Branching Strategy

SmartStock will follow **GitHub Flow with `develop` as the integration branch**. The provided development workflow specifies this approach for the module.

main
 │
 └── develop
     ├── feature/SS-XX-short-description
     ├── fix/SS-XX-short-description
     └── chore/SS-XX-short-description

Branch	Purpose
`main`	Stable and known-good version
`develop`	Integration branch where completed changes are combined
`feature/*`	New features or functionality
`fix/*`	Bug fixes
`chore/*`	Maintenance or non-feature tasks

The reference defines `main` as the known-good branch, `develop` as the integration branch, and `feature/*`, `fix/*`, and `chore/*` as ticket-based working branches.

Examples:

feature/SS-12-create-product
fix/SS-15-stock-validation
chore/SS-18-update-documentation

Each development branch should correspond to one Jira ticket. The reference also specifies a short branch lifetime, with feature/fix/chore branches intended to remain open for under three days.

3. Commit Message Convention

Commit messages must be precise, concise, and related to the Jira ticket.

Examples:

SS-12: add product creation
SS-13: fix stock validation

SS-14: update supplier listing
SS-15: update inventory form

Avoid:

update
changes
fix
final
final changes

The reference recommends making small commits and keeping the first line of the commit message concise.

4. Pull Request Process

A Pull Request is created after the assigned work has been implemented and locally tested.

Pull Request Flow

Development Complete
↓
Self-Review
↓
Create Pull Request
↓
Link Jira Ticket
↓
Include Acceptance Criteria
↓
Code Review
↓
SA Final Review
↓
Approval
↓
Squash Merge
↓
Delete Branch

Pull Request Requirements

The developer should:

1. Review their own changes before requesting review.
2. Create the Pull Request against `develop`.
3. Link the corresponding Jira ticket.
4. Include the acceptance criteria.
5. Address review comments.
6. Obtain the required approval.
7. Complete the SA final review before merging.

The reference requires Pull Requests to target `develop`, include the acceptance criteria, and be reviewed before squash merging and branch deletion.

5. Code Review Process

Before requesting review, the developer should review their own changes and ensure that the implementation satisfies the acceptance criteria.

Review Area	Description
Correctness	Check whether the implementation satisfies the acceptance criteria.
Boundaries	Check whether the implementation follows the expected module and technical boundaries.
Tests	Check whether appropriate tests are included or updated.
Security and Data	Check for security issues, data-access problems, or exposed secrets.

These four review areas are specified in the provided development workflow reference.

Label	Meaning
Blocking	Must be resolved before merge.
Suggestion	Recommended improvement.
Nit	Minor issue.

Question	Clarification is needed.

The reference specifically requires these labels so the author knows what prevents a merge and what does not.

6. SA Final Review

The **Software Architect** performs the final technical review before the implementation is merged.

The final review checks whether the implementation is consistent with the approved technical solution and architecture.

A change must not be merged while a blocking review comment remains unresolved. The reference also requires an approval from someone who did not write the code.

7. Feature Demonstration

After development and review, the developer demonstrates the completed feature or change.

The demonstration should show:

- What was implemented
- How the feature works
- How it satisfies the acceptance criteria
- Any relevant issue or limitation

Example

For a SmartStock product-management feature:

Open Product Management
↓
Select Add Product
↓
Enter Product Information
↓
Save Product
↓
Verify Product in Inventory

8. Bug Fix Workflow

SmartStock bug fixes will follow a separate fix process.

QA Identifies Bug

↓

Create Bug Ticket

↓

Create Fix Branch

↓

Write/Update Failing Test

↓

Implement Fix

↓

Local Testing

↓

Pull Request

↓

Code Review

↓

SA Final Review

↓

Merge

↓

QA Verification

↓

QA Closes Ticket

For a bug fix, the reference requires the failing test to be checked before the fix is accepted, and QA verifies and closes the ticket rather than the developer who wrote the fix.

9. Merge Rules

Changes may be merged only after the required review and approval have been completed.

The following rules apply:

Rule	Requirement
Branch protection	Direct pushes to protected branches are not allowed.
Pull Request	Changes must be submitted through a Pull Request.

Approval	At least one approval is required from someone who did not write the code.
Blocking comments	All blocking comments must be resolved before merging.
Merge method	Squash merge is used.
Branch cleanup	The working branch is deleted after merging.

These rules are based on the supplied development-process reference.

10. Development Workflow Summary

Refined Jira Ticket
↓
Requirement Review
↓
Create Branch from develop
↓
Development
↓
Local Testing
↓
Commit Changes
↓
Pull Request
↓
Code Review
↓
SA Final Review
↓
Feature Demonstration
↓
Approval
↓
Squash Merge

↓

QA Testing

↓

Done

SmartStock Technical Context

SmartStock follows a three-layer architecture consisting of **Frontend, Backend, and Database**. The current technical documentation identifies **Render** for backend hosting/deployment, **Supabase** for the database, and **GitHub** for source-code management.